# Van Gogh Classification and Neural Style Transfer

*Final Project* - Introduction to Deep Learning


**By:**

Matan Bar Tov | Omri Yarkoni

## Introduction

*This report analyzes a two-part deep learning pipeline built on a Post-Impressionism painting dataset: (i) fine-tuning two pretrained convolutional networks (VGG-19 and AlexNet) to perform binary classification of "Van Gogh" versus "Not Van Gogh," and (ii) implementing neural style transfer to generate Van Gogh-stylized images whose quality is evaluated using the trained classifiers. In addition, the report includes a CPU versus GPU runtime benchmark and a Grad-CAM-based interpretability analysis that probes which visual regions drive the classifier's decisions, followed by a conceptual proposal for using such heatmaps to improve the style transfer objective.*

## Part 1: Van Gogh classifier - discussion and analysis

### 1.1 Dataset overview

The classifier is trained on **Post-Impressionism paintings from the WikiArt dataset**, a large online collection of digitized artworks commonly used in computer vision research, where each image is associated with metadata such as artist and style. In the notebook, labels are loaded from *classes.csv*, images are matched from the folder *data/Post_Impressionism*, and the binary target is constructed as *is_van_gogh = 1* if *artist == "vincent van gogh"*, otherwise *0*. The filtered dataset contains **6,433** images in total, of which **1,004** are Van Gogh and **5,429** are non Van Gogh, so Van Gogh paintings form **15.6%** of the dataset. The provided split field subset is used to separate **train_full** (5,162 images) from **test** (1,271 images), and then the training set is further split into **train** (4,129 images) and **validation** (1,033 images) using a stratified 80/20 split. As shown in **Figure 1**, class proportions remain consistent across splits: train_full includes **807** Van Gogh images (15.6%), validation includes **161** (15.6%), and test includes **197** (15.5%).

| is_van_gogh | # train_full | # train | # val | # test |
|---|---|---|---|---|
| 0 | 4355 | 3483 | 872 | 1074 |
| 1 | 807 | 646 | 161 | 197 |
| total | 5162 | 4129 | 1033 | 1271 |

**Figure 1.** *Class distribution across dataset splits.*

### 1.2 Models and transformations

#### 1.2.1 Model architectures and transfer learning setup

Two pretrained convolutional neural networks are used as classification backbones: **AlexNet** and **VGG-19**, both imported from *torchvision.models* with ImageNet-pretrained weights (*pretrained=True*). **AlexNet** is an early, relatively compact CNN with **five convolutional layers** followed by **three fully connected layers**, and it was designed around aggressive downsampling and larger receptive fields in the early layers. In contrast, **VGG-19** is substantially deeper, with

**16 convolutional layers** and **3 fully connected layers**, and it relies on a design of **stacked 3×3 convolutions** with periodic max-pooling to build progressively more expressive features. This depth difference is a key practical distinction in this project: VGG-19 can represent **finer-grained hierarchical patterns** (texture, brushstroke-like micro-structures, composition cues) but typically requires **higher compute** and **memory cost**, whereas AlexNet is **faster** and lighter but **may underfit** subtle stylistic cues relative to VGG-19.
Both models are pretrained on **ImageNet**, meaning their initial feature extractors encode general-purpose visual primitives learned from natural images, which are then transferred to the WikiArt binary task. We apply a standard transfer learning strategy: the convolutional feature extractor (*model.features*) is frozen and the last classifier layer is replaced with a new Linear layer producing **two logits** (Van Gogh vs not Van Gogh), so training focuses on adapting the final decision boundary to the art domain while retaining the pretrained representation.

### 1.2.2 Preprocessing and data augmentations

All images are preprocessed to match the expectations of **ImageNet-pretrained** backbones. Inputs are normalized using the standard **ImageNet** statistics (mean = **[0.485, 0.456, 0.406]**, std = **[0.229, 0.224, 0.225]**), ensuring that pixel distributions are aligned with those seen during pretraining and stabilizing optimization when only the classifier head is fine-tuned. For training, the pipeline applies *RandomResizedCrop(224, scale=(0.8, 1.0))* to introduce moderate scale and framing variability, *RandomHorizontalFlip(p=0.5)* to encourage left-right invariance, and *ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.02)* to reduce sensitivity to digitization differences such as lighting, contrast, and color balance across artworks and scans. At **evaluation** time (validation and test), augmentation is removed and inputs are deterministically resized via *Resize((224, 224)),* followed by tensor conversion and the same ImageNet normalization, so reported performance reflects a consistent, repeatable preprocessing path.
A practical detail is the choice of **224×224 resizing for both models**. Historically, **AlexNet** is associated with **227×227** inputs in its original formulation, while **VGG** models use **224×224**. In this implementation, the torchvision AlexNet variant is compatible with 224×224 inputs, so **resizing to 224×224 works cleanly for both backbones** without requiring a model-specific preprocessing branch. Using a unified input size is important here because it keeps the comparison between AlexNet and VGG-19 focused on **architecture and learned representations**, rather than on differences introduced by the image pipeline.

## 1.3 Training and tuning

### 1.3.1 Training pipeline, optimization, and logging

Training is implemented in a modular pipeline that separates (i) **epoch-level optimization**, (ii) **validation-based model selection**, and (iii) **final evaluation**. During hyperparameter tuning, we train using *train_model_with_hyperparams*, which performs a standard supervised loop with **cross-entropy loss** (*nn.CrossEntropyLoss*) and **Adam** optimization (*optim.Adam*) on mini-batches from a *DataLoader*. Each epoch computes training loss and accuracy, then runs a full validation pass to compute validation loss and accuracy. Model selection is based on **minimum validation loss**: the best-performing weights across epochs are deep-copied and saved as a checkpoint for easy reproducibility. To reduce unnecessary computation, the training loop includes **early stopping** via *early_stop_check*, which stops training if validation loss does not improve for more than *patience* epochs. In parallel, the pipeline logs training and validation

metrics to **Weights and Biases** (*wandb.log*) at the end of each epoch, ensuring that learning dynamics can be inspected later when interpreting and diagnosing the Optuna results.

### 1.3.2 AlexNet Hyperparameter search setup with Optuna

For **AlexNet**, hyperparameter selection is performed with **Optuna** using **4-fold cross validation** (K-fold with K = 4) on the **train_full split**, meaning each trial trains and validates on four folds drawn from the larger training pool (5,162 images) rather than on a smaller single train/validation partition. This increases the amount of data used for fitting within each fold and yields a validation estimate that is less sensitive to one particular split, which is especially useful given the moderate class imbalance. Because each AlexNet Optuna trial includes **4 folds**, runtime grows quickly with the number of epochs and trials. In the project's setup, a single trial with **4-fold cross validation and 1 epoch** takes approximately **5.5 minutes**, so to keep the search computationally feasible while still exploring the space, the sweep budget is set to **5 trials** training each fold for **2 epochs** with early stopping (*patience == 1*) and **median pruning** (strategy that terminates a trial if its intermediate performance is worse than the median performance of previous trials at the same step) in order to optimize the runtime of the code. This choice prioritizes breadth of exploration under strict time constraints: the goal is to identify a reasonable region of learning rate, weight decay, and batch size that already yields strong validation performance, and then rely on the final training stage (outside the sweep) to further refine the model with the selected hyperparameters.

The AlexNet search space is designed for a transfer-learning setting where only the classifier head is optimized: *learning_rate* is sampled log-uniformly in **[1e-5, 1e-3]** to cover conservative to moderately aggressive step sizes for Adam. *weight_decay* is sampled log-uniformly in **[1e-6, 1e-4]** to provide mild regularization without over-penalizing the newly initialized head. and *batch_size* is sampled from **{16, 32, 48, 64}** to balance gradient noise, stability, and throughput while staying within typical memory limits.

### 1.3.3 AlexNet Optuna results and selected hyperparameters

The AlexNet Optuna sweep evaluated **5 trials** under the **4-fold cross validation objective** described above, and selected the final configuration by minimizing the reported **validation loss** (Figure 2). The total Runtime of the Optuna function was around 55 minutes. The best configuration is **AlexNet_trial_1**, achieving **validation loss = 0.17423** with **validation accuracy = 0.93953**, while also reaching **train loss = 0.15908** and **train accuracy = 0.93905** in the W&B summary table. The corresponding hyperparameters are **batch_size = 64**, **learning_rate = $6.99908\times10^{-5}$**, and **weight_decay = $1.0102\times10^{-5}$**. Compared to the remaining trials, this best setting combines the largest batch size with a relatively small learning rate. notably, the trial with a much larger learning rate (**$7.8465\times10^{-4}$**, AlexNet_trial_0) attains a substantially worse **validation loss = 0.24118**, consistent with the expectation that overly aggressive step sizes can harm generalization even when the training objective remains learnable. The best-trial training curves (Figure 3) show a zigzag pattern across logged steps, which is expected under the K-fold protocol because metrics are logged repeatedly across different folds and epochs, producing visible fold-to-fold variability while maintaining strong overall performance.

| NAME 5 visualized | STATE | ARCHITECTURE | RUNTIME | TRAIN ACCURACY | TRAIN LOSS | VALIDATION ACCURACY | VALIDATION LOSS | BATCH_SIZE | LEARNING_RATE | WEIGHT_DECAY |
|---|---|---|---|---|---|---|---|---|---|---|
| AlexNet_trial_4 | Finished | AlexNet | 5m 13s | 0.89098 | 0.27811 | 0.92486 | 0.1904 | 16 | 0.000056507 | 0.0000013604 |
| AlexNet_trial_3 | Finished | AlexNet | 5m 13s | 0.88065 | 0.30972 | 0.91325 | 0.21701 | 16 | 0.00001674 | 0.0000011142 |
| AlexNet_trial_2 | Finished | AlexNet | 5m | 0.87833 | 0.31642 | 0.91325 | 0.2162 | 32 | 0.000026157 | 0.000059876 |
| AlexNet_trial_1 | Finished | AlexNet | 19m 41s | 0.93905 | 0.15908 | 0.93953 | 0.17423 | 64 | 0.000069908 | 0.000010102 |
| AlexNet_trial_0 | Finished | AlexNet | 19m 50s | 0.91219 | 0.2339 | 0.90775 | 0.24118 | 64 | 0.00078465 | 0.0000015484 |

**Figure 2.** *Weights and Biases summary table for AlexNet hyperparameter sweep trials.*


**Figure 3.** *AlexNet best trial (AlexNet_trial_1) learning curves (W&B).*

### 1.3.4 VGG Hyperparameter search setup with Optuna

For **VGG-19**, hyperparameter selection is performed with **Optuna** using a **fixed train/validation split** (no K-fold cross validation). This choice is primarily driven by runtime: in preliminary timing, a single VGG-19 trial with **4-fold cross validation and 1 epoch** took roughly **11 minutes**, which would make a multi-trial sweep overly expensive once folds and additional epochs are included. Consequently, we decided to adopt a standard **train/validation split** strategy and allocate the tuning budget to exploring more configurations and more epochs per configuration, rather than multiplying computation by folds.

Concretely, each Optuna trial samples a hyperparameter configuration, initializes an ImageNet-pretrained VGG-19 backbone converted to a binary classifier head (frozen model.features with a replaced final Linear layer), trains on *train_dataset*, and evaluates on *val_dataset*. The trial objective is the **best validation loss** achieved during training, and each trial is logged to **Weights and Biases** to enable inspection of train and validation curves. The VGG-19 search space is chosen to reflect the higher sensitivity and compute cost of a deeper model: *learning_rate* is sampled log-uniformly in **[1e-6, 3e-4]**, *weight_decay* is sampled log-uniformly in **[1e-6, 1e-3]**, and *batch_size* is sampled from **{16, 32, 64}**. Under this runtime-aware design, the sweep budget is set to **6 trials** with **3 epochs per trial** with early stopping (*patience* == *2*) and **median pruning**, which provides a reasonable compromise between exploring the hyperparameter space and allowing each configuration enough training steps to show meaningful validation trends.

### 1.3.5 VGG Optuna results and selected hyperparameters

The VGG-19 Optuna sweep evaluated **6 trials** under the train/validation objective described above, and selected the final configuration by minimizing **validation loss** (Figure 4). The total runtime of the Optuna function was around 38 minutes. The best configuration is **VGG_trial_5**, achieving **validation loss = 0.15345** with **validation accuracy = 0.94676**, while also reporting **train loss = 0.14143** and **train accuracy = 0.94333** in the W&B sweep summary table. The corresponding hyperparameters are **batch_size = 64**, **learning_rate = $3.3025\times10^{-5}$**, and **weight_decay = $1.6222\times10^{-6}$**. Relative to the other trials, the results highlight that learning rate is a key driver under the short trial budget: extremely small learning rates (for example *VGG_trial_2* with **$1.7774\times10^{-6}$**) yield substantially worse validation loss (**0.40009**), consistent with under-updating within only three epochs, while larger learning rates remain competitive but do not achieve the best loss in this setup (for example *VGG_trial_1* with **$2.3686\times10^{-4}$** reaches validation loss **0.17264**). The best-trial curves (Figure 5) show a stable training trajectory across logged steps, with both training and validation losses decreasing and both accuracies increasing, supporting that the selected hyperparameters provide reliable convergence under the chosen computational budget.

| NAME (6 visualized) | STATE | ARCHITECTURE | RUNTIME | TRAIN ACCURACY | TRAIN LOSS | VALIDATION ACCURACY | VALIDATION LOSS | BATCH_SIZE | LEARNING_RATE | WEIGHT_DECAY |
|---|---|---|---|---|---|---|---|---|---|---|
| VGG_trial_5 | Finished | VGG19 | 7m 13s | 0.94333 | 0.14143 | 0.94676 | 0.15345 | 64 | 0.000033025 | 0.0000016222 |
| VGG_trial_4 | Finished | VGG19 | 4m 46s | 0.84791 | 0.37778 | 0.87803 | 0.28867 | 64 | 0.000015307 | 0.000001147 |
| VGG_trial_3 | Finished | VGG19 | 5m 17s | 0.83846 | 0.38717 | 0.87803 | 0.29843 | 16 | 0.0000060742 | 0.000024982 |
| VGG_trial_2 | Finished | VGG19 | 4m 52s | 0.79365 | 0.50213 | 0.84414 | 0.40009 | 32 | 0.0000017774 | 0.0000011167 |
| VGG_trial_1 | Finished | VGG19 | 7m 12s | 0.95592 | 0.11578 | 0.94095 | 0.17264 | 64 | 0.00023686 | 0.000019016 |
| VGG_trial_0 | Finished | VGG19 | 8m 2s | 0.95665 | 0.11439 | 0.93998 | 0.1746 | 16 | 0.00007187 | 0.000001562 |

**Figure 4.** *Weights and Biases summary table for VGG-19 hyperparameter sweep trials.*

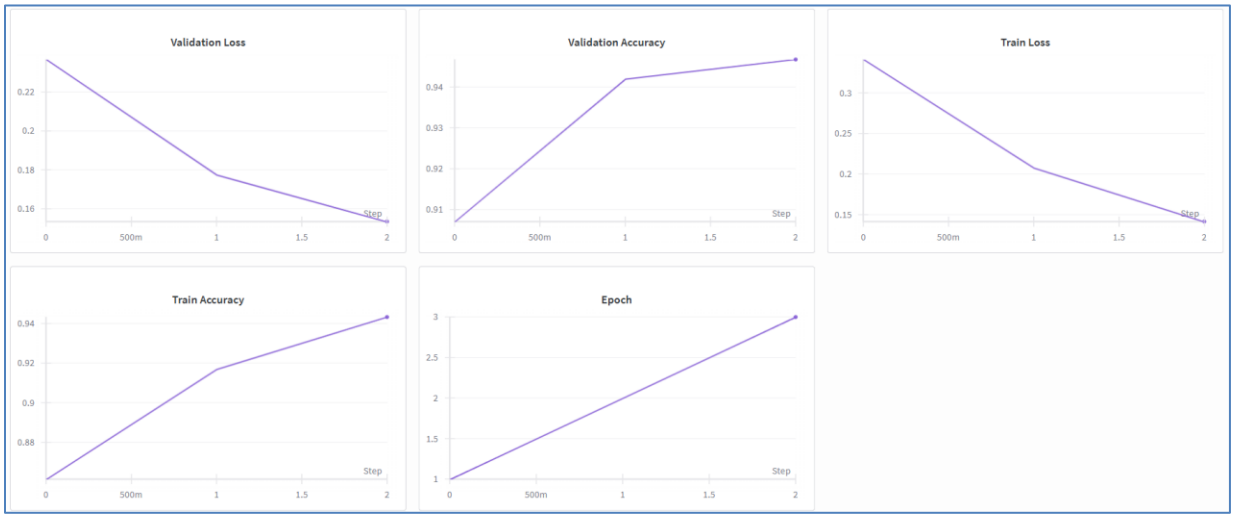


**Figure 5.** *Weights and Biases learning curves for the best VGG-19 sweep trial.*

### 1.3.6 AlexNet Final training using the selected hyperparameters

After selecting the best AlexNet hyperparameters via Optuna, the model runs a **final training stage** with 10 epochs using the chosen configuration (**batch_size = 64**, **learning_rate = $6.9908\times10^{-5}$**, **weight_decay = $1.0102\times10^{-5}$**) and logs epoch-level metrics to W&B. A single epoch takes approximately 3.5 minutes , so the number of epochs is chosen so that the model optimizes but also takes into account a reasonable runtime. The **total runtime** of the final training was **35 minutes**. Figure 6 shows that training behavior is well-conditioned: train loss decreases steadily while train accuracy, AUC, precision, recall, and F1 increase and then plateau, indicating fast convergence consistent with the transfer-learning regime (frozen feature extractor and an optimized classifier head). Figure 7 reports the corresponding **test-set** metrics monitored across training steps. As an example, at **Step 1** the test metrics are: **accuracy = 0.9465**, **AUC = 0.97153**, **F1 = 0.81622**, **precision = 0.87283**, **recall = 0.7665**, and **test loss = 0.1479**. Across the logged steps, test accuracy stays in a narrow band around the mid-0.94 range, while recall varies more strongly than precision, which is consistent with the dataset's class imbalance where sensitivity to the minority class (Van Gogh) is typically the most volatile component.

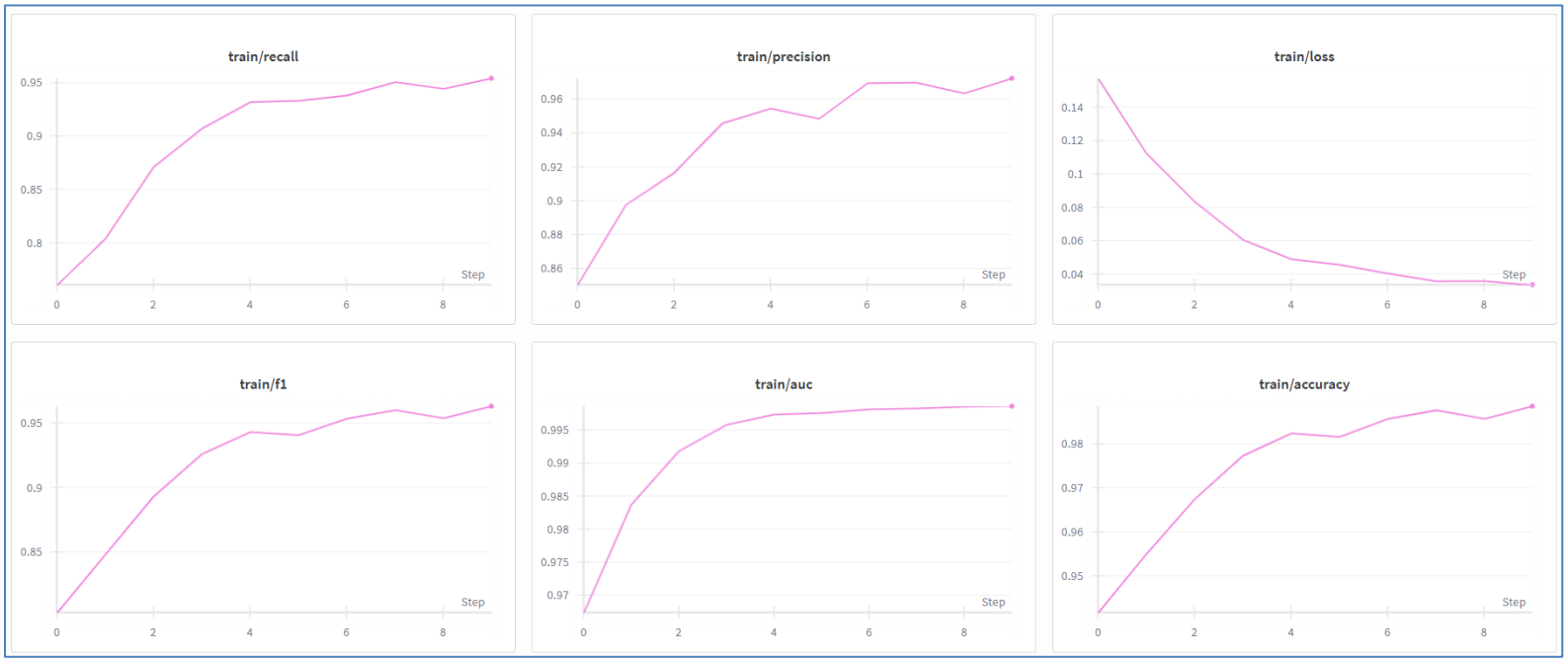


**Figure 6.** *AlexNet final **training metrics** (W&B): train loss, accuracy, AUC, F1, precision, and recall across training steps.*

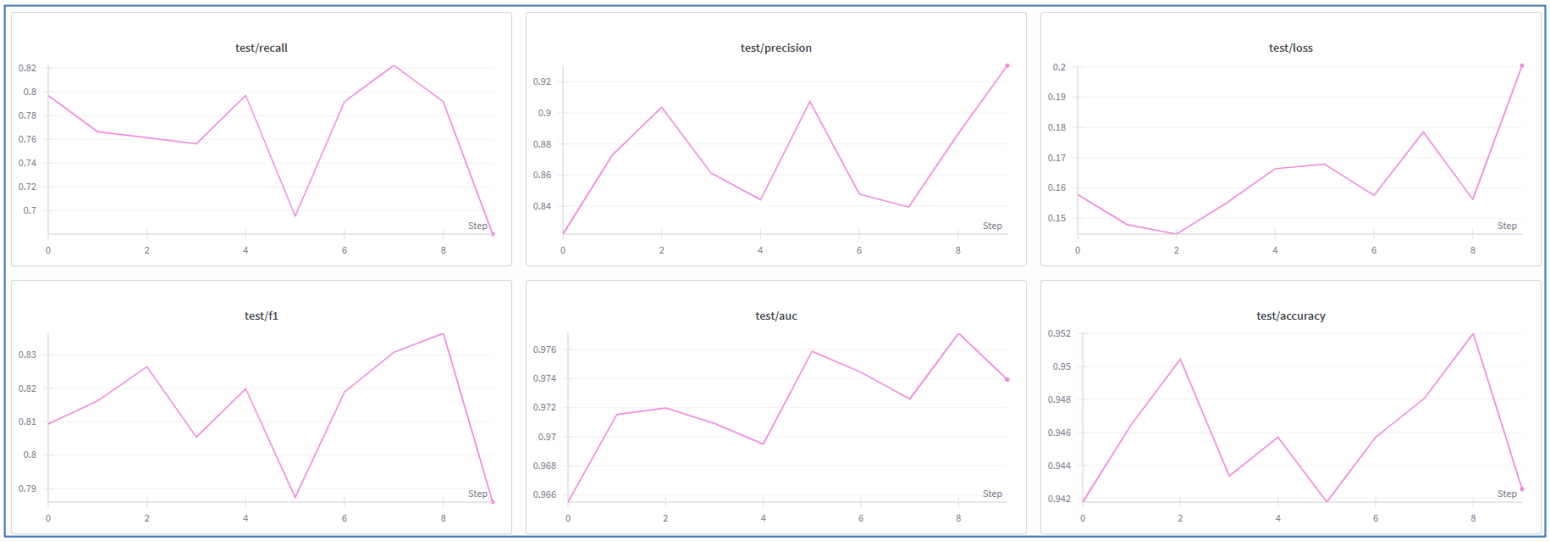


**Figure 7.** *AlexNet final **test metrics** (W&B): test loss, accuracy, AUC, F1, precision, and recall across training steps.*

### 1.3.7 VGG Final training using the selected hyperparameters

After selecting the best VGG-19 hyperparameters via Optuna, the project runs a **final training stage** using the chosen configuration (**batch_size = 64**, **learning_rate = $3.3025\times10^{-5}$**, **weight_decay = $1.6222\times10^{-6}$**) and logs epoch-level metrics to W&B. The total runtime of the final training was 32 minutes. Figure 8 shows consistent optimization on the training set: train loss decreases smoothly while train accuracy and AUC rapidly rise and then saturate near 1.0, and the positive-class precision, recall, and F1 improve throughout training. Figure 9 summarizes test-set behavior across the same steps. Test accuracy remains relatively stable (around the mid-0.94 range), while AUC stays high (roughly mid-0.95 to low-0.96), indicating strong ranking ability even when threshold-dependent metrics (recall and F1) fluctuate more. The test loss curve increases during later steps before decreasing again, which suggests that continued optimization primarily changes confidence calibration rather than improving separability, a common effect when training a classifier head beyond the point of peak generalization.

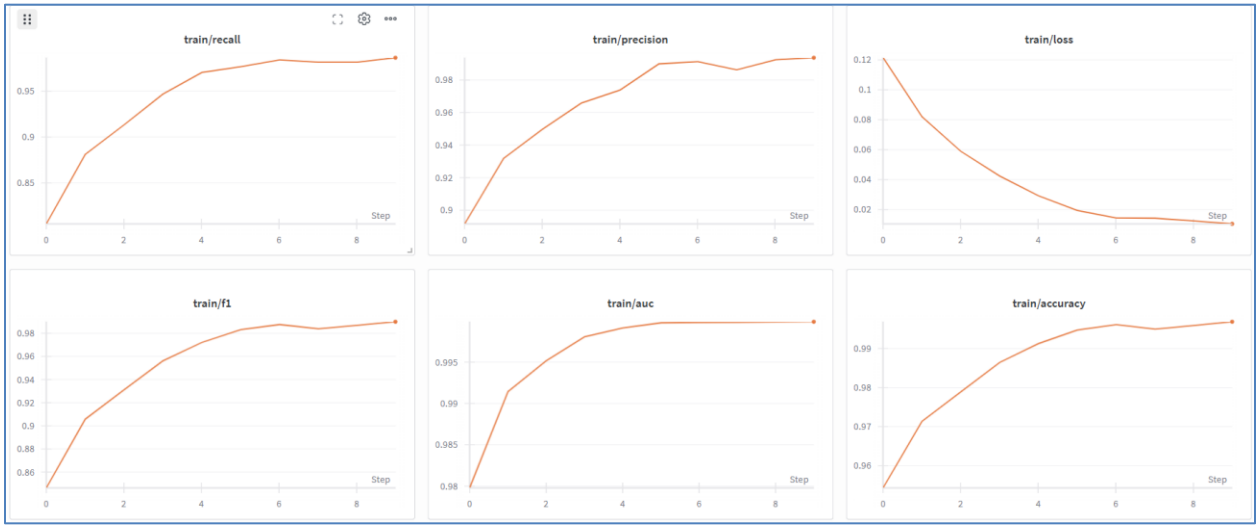


**Figure 8.** *VGG-19 final **training metrics** (W&B): train loss, accuracy, AUC, F1, precision, and recall across training steps.*

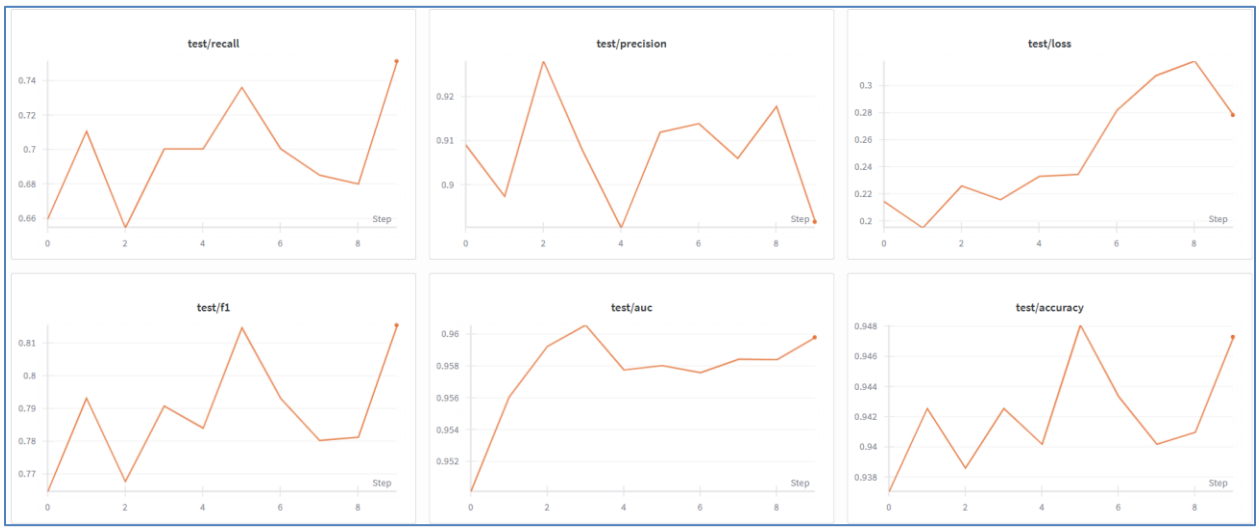


**Figure 9.** *VGG-19 final **test metrics** (W&B): test loss, accuracy, AUC, F1, precision, and recall across training steps.*

## 1.4 Model comparison

On the **test set**, the two models exhibit a clear tradeoff between **ranking performance** and **minority-class sensitivity**. AlexNet achieves **Accuracy = 0.9426**, **AUC-ROC = 0.9739**, **F1 = 0.7859**, **Precision = 0.9306**, **Recall = 0.6802**, while VGG-19 achieves **Accuracy = 0.9473**, **AUC-ROC = 0.9598**, **F1 = 0.8154**, **Precision = 0.8916**, **Recall = 0.7513**. These differences are summarized visually in **Figure 10**, which highlights that VGG-19 performs better in **accuracy, recall, and F1**, indicating stronger detection of the Van Gogh class (higher recall) at the cost of more false positives (lower precision). In contrast, AlexNet attains **higher AUC** and **higher precision**, suggesting stronger overall separability in terms of score ranking and a more conservative positive prediction threshold under the evaluated decision rule. This ranking difference is also illustrated directly by the **test ROC curves in Figure 11** (AUC = 0.9739 for AlexNet vs 0.9598 for VGG-19). A plausible explanation is architectural: VGG-19's depth and repeated 3×3 convolutions can represent finer-grained hierarchical texture cues that help recover more true Van Gogh instances, while AlexNet's lower capacity may yield a sharper but more conservative decision boundary that reduces false positives and improves precision, even when recall is lower. These findings motivate using **F1 and recall** (not accuracy alone) as primary comparison criteria in this imbalanced setting, because the Van Gogh class is the minority.

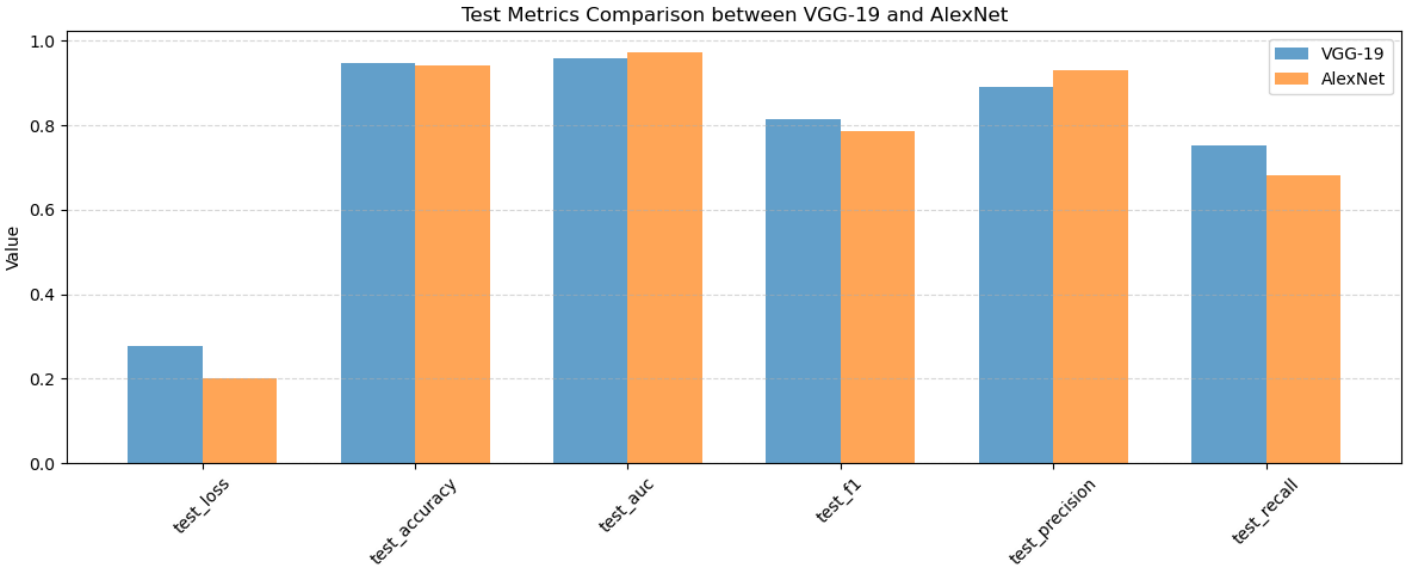


**Figure 10.** *Comparison of AlexNet and VGG-19 test-set performance metrics (Accuracy, AUC-ROC, F1, Loss, Precision, Recall).*

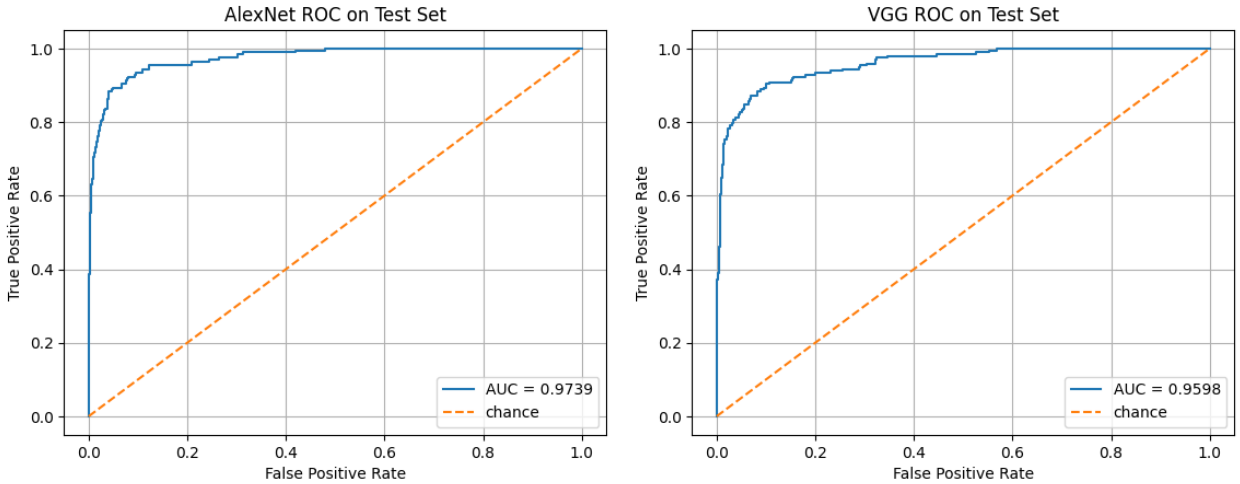


**Figure 11.** *Test-set ROC curves for AlexNet (AUC = 0.9739) and VGG-19 (AUC = 0.9598).*

## 1.5 Prediction analysis

### 1.5.1 Confusion matrices

Figures 12–13 report the confusion matrices of AlexNet and VGG-19 on the **test set (N = 1,271)**, which contains **1,074 non Van Gogh** and **197 Van Gogh** paintings. For **AlexNet** (Figure 12), the model predicts **1064** true negatives and **134** true positives, with **10** false positives and **63** false negatives. For **VGG-19** (Figure 13), the model predicts **1056** true negatives and **148** true positives, with **18** false positives and **49** false negatives. This aligns with the metric-level comparison in Section 1.4: VGG-19 has higher **recall** (fewer missed Van Gogh paintings: 49 vs 63), while AlexNet has higher **precision** (fewer non Van Gogh paintings incorrectly flagged as Van Gogh: 10 vs 18). VGG-19 also achieves a higher F1 because the gain in recall outweighs the accompanying drop in precision relative to AlexNet. In practical terms, VGG-19 is more sensitive to Van Gogh style cues but pays for that sensitivity with more false alarms, whereas AlexNet is more conservative in predicting the positive class.

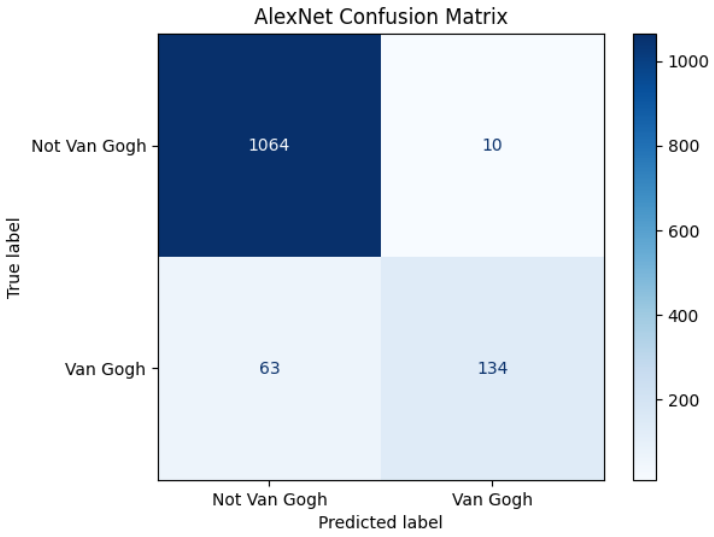


**Figure 12.** *AlexNet confusion matrix (rows: true labels, columns: predicted labels).*

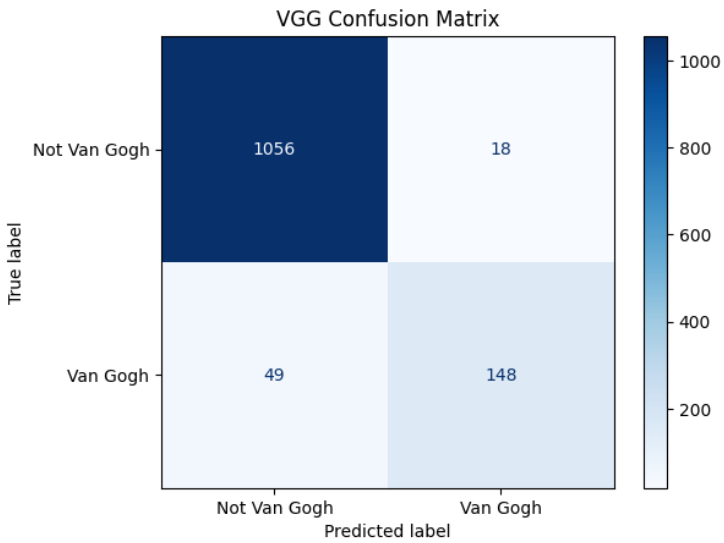


**Figure 13.** *VGG-19 confusion matrix (rows: true labels, columns: predicted labels).*

### 1.5.2 AlexNet examples of TP, TN, FP, and FN

Figure 14 presents **True Positives (TP)** for AlexNet, where Van Gogh paintings are correctly classified as Van Gogh with high confidence (several examples show **P(Van Gogh) = 1.00**, and one more ambiguous case appears near the decision boundary with **P(Van Gogh) = 0.51**). In these TP examples, the model succeeds on works dominated by highly salient "Van Gogh-like" cues in this dataset, including dense directional stroke texture in landscapes, as well as strong line-based structure in drawings or monochrome sketches, suggesting the classifier is not relying on color alone. Figure 15 shows **True Negatives (TN)**, where non Van Gogh paintings are correctly rejected with extremely low predicted probability (**P(Van Gogh) = 0.00** for all displayed samples). These TN cases span diverse non Van Gogh styles, and the consistently near-zero probabilities indicate that AlexNet learned a decision rule that confidently separates many non Van Gogh paintings from the Van Gogh class in the test set.

The error cases reveal what the model confuses with Van Gogh. Figure 16 shows **False Positives (FP)**: non Van Gogh paintings classified as Van Gogh, sometimes with very high confidence (for example **P(Van Gogh) = 0.96** and **0.98**). Visually, these FPs often contain strong post-impressionist-like texture and color cues, such as vivid blues, thick brushstroke-like patterns, and high-frequency directional marks, which can resemble the textures of Van Gogh paintings and lead the model to overgeneralize. Figure 17 shows **False Negatives (FN)**: true Van Gogh paintings missed by the model, typically with **very low P(Van Gogh)** (several examples are near **0.00–0.09**, with one less certain miss at **0.38**). In these FN examples, the Van Gogh works appear to deviate from the most stereotypical cues the classifier relies on, for instance when the composition is dominated by smoother regions, less pronounced stroke texture, or style elements that resemble broader post-impressionist patterns shared across artists. Importantly, the displayed probabilities indicate that many FNs are not borderline mistakes but confident rejections, which

is consistent with AlexNet's overall tendency toward **high precision and lower recall** relative to VGG-19 in Section 1.4.

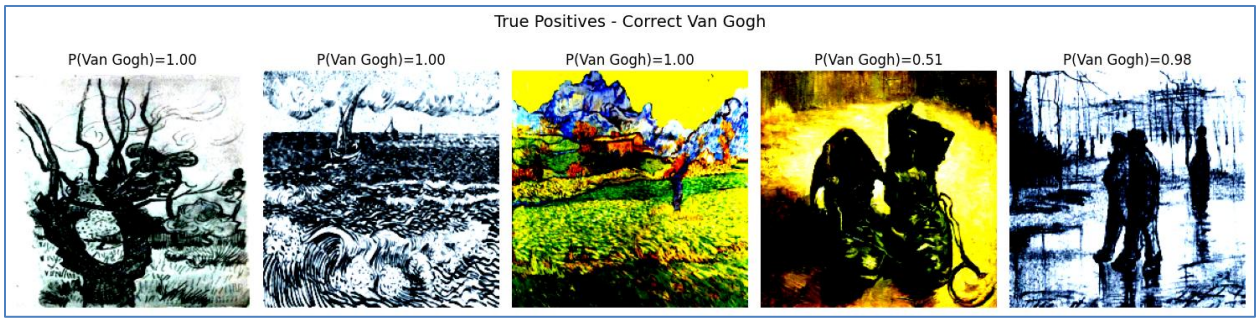


**Figure 14.** *AlexNet True Positives: correctly classified Van Gogh paintings.*


**Figure 15.** *AlexNet True Negatives: correctly classified non Van Gogh paintings.*


**Figure 16.** *AlexNet False Positives: non Van Gogh paintings misclassified as Van Gogh.*


**Figure 17.** *AlexNet False Negatives: Van Gogh paintings misclassified as non Van Gogh.*

### 1.5.3 VGG-19 examples of TP, TN, FP, and FN

Figure 18 presents **True Positives (TP)** for VGG-19, where Van Gogh paintings are correctly classified as Van Gogh with high confidence in most cases (**P(Van Gogh) = 1.00** appears multiple times), alongside moderately confident positives (**P(Van Gogh) = 0.82** and **0.69**). Compared to AlexNet, these examples suggest that VGG-19 maintains correct positive predictions even when the visual evidence is less extreme, consistent with its higher recall. Figure 19 shows **True Negatives (TN)**, where non Van Gogh works are correctly rejected with near-zero predicted probability (**P(Van Gogh) = 0.00** for most displayed samples, and a near-zero case at **0.01**), indicating that VGG-19 still preserves strong specificity on many non Van Gogh styles despite being more sensitive overall.

The error modes highlight the boundary cases where VGG-19 overgeneralizes or misses atypical Van Gogh works. Figure 20 shows **False Positives (FP)**, where non Van Gogh paintings are classified as Van Gogh, often with very high confidence (**P(Van Gogh) = 0.95**, **0.99**, and **1.00**). Similar to AlexNet, these FPs include paintings with heavy stroke-like texture and saturated chromatic contrasts. Additionally, the presence of a highly confident line-drawing-like FP (the elephant sketch at **P(Van Gogh) = 1.00**) suggests that VGG-19 may associate strong black-and-white contour structure with the positive class when such structure resembles Van Gogh drawings in the training set. Figure 21 shows **False Negatives (FN)**, namely Van Gogh paintings missed by the model. In this grid, several misses have **P(Van Gogh) = 0.00**, indicating confident rejections rather than borderline cases, and one miss appears closer to the boundary (**P(Van Gogh) = 0.37**). Visually, these FNs again appear to correspond to Van Gogh works whose cues are less aligned with the dominant "Van Gogh signature", such as atypical color composition, local regions with smoother texture, or structural patterns that overlap with broader post-impressionist content. Overall, the patterns are consistent with Section 1.4: VGG-19 recovers more positives than AlexNet but can be more prone to texture-driven overgeneralization, producing high-confidence false positives in visually similar non Van Gogh works.


**Figure 18.** *VGG-19 True Positives: correctly classified Van Gogh paintings.*


**Figure 19.** *VGG-19 True Negatives: correctly classified non Van Gogh paintings.*


**Figure 20.** *VGG-19 False Positives: non Van Gogh paintings misclassified as Van Gogh.*


**Figure 21.** *VGG-19 False Negatives: Van Gogh paintings misclassified as non Van Gogh.*

### 1.5.4 Summary of error patterns and implications

Across both models, the grids indicate that the classifier decision is strongly influenced by **stroke and texture statistics** as well as **high-contrast contour structure**, not only by global subject matter. In particular, many **false positives** from both AlexNet and VGG-19 exhibit visually salient brushstroke-like patterns and saturated chromatic contrasts that resemble Van Gogh's post-impressionist texture signature, while several **false negatives** correspond to Van Gogh works that appear stylistically atypical or that contain weaker high-frequency stroke cues. Comparing the two, VGG-19 tends to recover more true Van Gogh examples (higher recall), but it also shows more high-confidence false positives on non Van Gogh paintings that share these texture and contour cues, consistent with the precision-recall tradeoff in Section 1.4. These observations motivate the analysis in Part 2: because neural style transfer explicitly manipulates texture and style statistics, it provides a controlled way to test whether inducing Van Gogh-like texture in non Van Gogh content images is sufficient to flip the classifier decision, and whether AlexNet and VGG-19 respond similarly to these style-driven perturbations.

## Part 2: Recreating Van Gogh's Style - Neural Style Transfer

### 2.1 Style Transfer Process

#### 2.1.1 Neural style transfer objective

Neural style transfer formulates image stylization as an optimization problem over the pixels of a target image $x$, given a **content** image $c$ and a **style** image $s$. A fixed pretrained convolutional network $\phi(\cdot)$ is used purely as a feature extractor. For any layer $\ell$, let $\phi_\ell(x) \in \mathbb{R}^{C_\ell \times H_\ell \times W_\ell}$ denote the activation tensor of $x$ at that layer, and let $F_\ell(x) \in \mathbb{R}^{C_\ell \times N_\ell}$ be the flattened feature map where $N_\ell = H_\ell W_\ell$.

**Content preservation** is enforced by matching deep feature activations between the target and the content image at a set of content layers $\mathcal{L}_c$:

$$\mathcal{L}_{\text{content}}(x, c) = \sum_{\ell \in \mathcal{L}_c} \| \phi_\ell(x) - \phi_\ell(c) \|_2^2 .$$

**Style matching** is enforced by matching feature correlations, commonly captured by **Gram matrices**. For layer $\ell$, the Gram matrix is:

$$G_\ell(x) = \frac{1}{C_\ell N_\ell} F_\ell(x)\, F_\ell(x)^\top \in \mathbb{R}^{C_\ell \times C_\ell},$$

which summarizes how strongly feature channels co-activate, largely discarding spatial arrangement. Using a set of style layers $\mathcal{L}_s$, the style loss is:

$$\mathcal{L}_{\text{style}}(x, s) = \sum_{\ell \in \mathcal{L}_s} w_\ell \| G_\ell(x) - G_\ell(s) \|_F^2,$$

where $w_\ell$ are nonnegative layer weights (often normalized to sum to 1). The final objective trades off content fidelity and style strength:

$$\mathcal{L}(x) = \alpha\, \mathcal{L}_{\text{content}}(x, c) + \beta\, \mathcal{L}_{\text{style}}(x, s),$$

with $\alpha$ and $\beta$ corresponding to *content_weight* and *style_weight* in our code implementation. Increasing $\alpha$ forces the output to more closely preserve the content structure (often at the expense of weaker stylization), while increasing $\beta$ strengthens style texture transfer but can distort content geometry if set too high. The stylized image is obtained by gradient-based optimization:

$$x^\star = \arg\min_x \mathcal{L}(x),$$

typically initialized from the content image and optimized with an algorithm such as Adam.

A key practical aspect is the role of network depth. **Early convolutional layers** (closer to the input) encode local edges, color gradients, and fine textures with high spatial resolution, matching their Gram matrices tends to transfer **low-level texture and brushstroke statistics**. **Deeper layers** encode more abstract and larger-scale patterns (shapes, object parts, and global composition) with more invariance to small pixel changes. Using deeper layers for content matching helps preserve **semantic structure and layout** of the content image. Style transfer leverages this separation: style is often captured by correlations across multiple early-to-mid layers, while content is preserved using one or more deeper layers.

### 2.1.2 Mapping the theory to the project implementation

Figure 22a–22c presents the full *style_transfer* implementation used in this project, together with the helper functions that support feature extraction, Gram computation, normalization, and visualization (Figure 23a–23c). The function receives a pretrained CNN backbone (model), a content image $c$, a style image $s$, and the loss tradeoff parameters $\alpha =$*content_weight* and $\beta =$*style_weight* , and then performs gradient-based optimization directly over a target image $x$. Concretely, the backbone is restricted to its convolutional feature extractor via *model = model.features* , moved to device, set to evaluation mode, and frozen by disabling gradients for its parameters (*param.requires_grad_(False)*), ensuring that only the target pixels are optimized (Figure 22b).

**Layer selection** - If the user does not provide *content_layers* and *style_layers*, the function uses backbone-specific defaults (Figure 22a):

- **AlexNet:** *style_layers* = ['0','3','6','8'], *content_layers* = ['10']
- **VGG:** *style_layers* = ['0','5','10','19'], *content_layers* = ['21']

This matches the standard neural style transfer intuition described in Section 2.1.1: style is extracted from multiple earlier-to-mid layers (capturing texture and stroke statistics at different scales), while content is extracted from a deeper layer (capturing higher-level spatial structure). The code also supports explicit per-style-layer weights through *style_layer_weights*, if not provided or mismatched in length, the function uses uniform weights, otherwise it normalizes the provided weights to sum to 1 (Figure 22a).

**Feature extraction and Gram statistics** - The helper *get_features(image, model, layers)* iterates through *model._modules.items()* and stores activations for the requested layer names (Figure 23b). The Gram matrix is computed by reshaping a feature tensor of shape $(N, C, H, W)$ to *(N,C,HW)* , multiplying $xx^\top$, and normalizing by $(C \cdot H \cdot W)$ (Figure 23b), which corresponds

directly to the $G_\ell(\cdot)$ definition in Section 2.1.1 and implements a scale-stabilized style statistic. Images are loaded with load_image, which resizes to *(224,224)* , converts to tensor, and applies ImageNet normalization, returning a batched tensor on device (Figure 23b). The *denormalize_image* helper reverses this normalization for visualization and saving (Figure 23b), while *to_display_01* and *show_tensor_image* enforce clamping to $[0,1]$ and proper channel ordering for matplotlib (Figure 23a). For qualitative presentation, *show_triplet* displays the content, style, and output side-by-side (Figure 23c).

**Optimization loop and valid-range constraints -** The target image is initialized as a clone of the content image (*target = content.clone().to(device)* and *target.requires_grad_(True)*), implementing the common "content initialization" strategy (Figure 22b). Since all tensors remain in normalized ImageNet space during optimization, the code computes per-channel lower and upper bounds corresponding to pixel intensities $[0,1]$ after normalization:

$$\text{low\_bound} = \frac{0-\mu}{\sigma}, \quad \text{high\_bound} = \frac{1-\mu}{\sigma} \ ,$$

and clamps the target to this range after each update (*target.clamp_(low_bound, high_bound)*), preventing the optimizer from producing invalid normalized values (Figure 22b–22c). Style Gram matrices are computed once from the fixed style features, and at each iteration the function recomputes target features and accumulates: (i) **content loss** as the sum of MSE losses between target and content feature maps at the chosen *content_layers*, and (ii) **style loss** as the weighted sum of MSE losses between Gram matrices of target and style at each chosen *style_layers* (Figure 22c). The total objective is exactly the weighted sum described in Section 2.1.1:

$$\mathcal{L}(x) = \alpha\, \mathcal{L}_{\text{content}}(x, c) + \beta\, \mathcal{L}_{\text{style}}(x, s),$$

implemented as *total_loss = content_weight * content_loss + style_weight * style_loss*, optimized using *optim.Adam([target], lr=0.003)* for *num_steps* iterations (default 500). The learning rate **0.003** is a pragmatic choice for pixel-space optimization with Adam: it is large enough to produce visible stylization progress within a few hundred steps (avoiding extremely slow convergence), yet typically small enough to keep the optimization stable when both content and Gram-based style losses are active, especially under the explicit per-step clamping that prevents the target from drifting into invalid normalized ranges. The learning rate is kept fixed (rather than tuned in Optuna) because it primarily affects optimization stability and convergence speed under a fixed step budget, and tuning it would increase sweep cost without proportionate gains in the stylization tradeoff.
Finally, the output is denormalized and clamped to $[0,1]$ before returning the output image and scalar loss values (Figure 22c).

```python
def style_transfer(model, content_image, style_image, content_weight, style_weight, num_steps=500, content_layers=None, style_layers=None, style_layer_weights=None):

    model_name = model.__class__.__name__
    model = model.features #Gives us access to the layers of features

    if (not content_layers) and (not style_layers):
        if model_name == 'AlexNet':
            style_layers = ['0', '3', '6', '8']
            content_layers = ['10']
        elif model_name == 'VGG':
            style_layers = ['0', '5', '10', '19']
            content_layers = ['21']


    if (style_layer_weights is None) or (len(style_layer_weights) != len(style_layers)):
        n_style = len(style_layers)
        style_layer_weights = [1.0 / n_style] * n_style
    else:
        # normalize weights to sum to 1
        w = torch.tensor(style_layer_weights, dtype=torch.float32, device=device)
        w = w / (w.sum())
        style_layer_weights = w.detach().cpu().tolist()

    layers = content_layers + style_layers

    content = content_image.to(device)
    style = style_image.to(device)
```

**Figure 22a.** *Code implementation of the style_transfer function, part (a)*

```python
    # Prepare model for evaluation, disabling gradient computation
    model.to(device).eval()
    for param in model.parameters():
        param.requires_grad_(False)
    # Extract features from content and style images
    content_features = get_features(content, model, layers)
    style_features = get_features(style, model, layers)
    target = content.clone().to(device)
    target.requires_grad_(True)

    # Define bounds for clamping the target image
    image_mean = torch.tensor(IMAGENET_MEAN, device=target.device).view(1,3,1,1)
    image_std  = torch.tensor(IMAGENET_STD, device=target.device).view(1,3,1,1)
    low_bound  = (0.0 - image_mean) / image_std
    high_bound = (1.0 - image_mean) / image_std


    style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_layers}
    optimizer = optim.Adam([target], lr=0.003)
    # Style transfer loop
    for i in range(1, num_steps + 1):
        # Extract features from target image
        target_features = get_features(target, model, layers)
        # Compute content loss
        content_loss = 0
        for layer in content_layers:
            content_loss += F.mse_loss(target_features[layer], content_features[layer])
        content_loss = content_loss / len(content_layers)
```

**Figure 22b.** *Code implementation of the style_transfer function, part (b)*

```
        # Compute style loss by comparing Gram matrices for each layer
        style_loss = 0
        for j, layer in enumerate(style_layers):
            target_feature = target_features[layer]
            target_gram = gram_matrix(target_feature)
            style_gram = style_grams[layer]
            style_loss += style_layer_weights[j] * F.mse_loss(target_gram, style_gram)
        # Calculate total loss and update target image
        total_loss = content_weight * content_loss + style_weight * style_loss
        optimizer.zero_grad()
        total_loss.backward()
        optimizer.step()

        # Clamp target image to valid range
        with torch.no_grad():
            target.clamp_(low_bound, high_bound)

    final_content_loss = content_loss
    final_style_loss = style_loss
    final_total_loss = total_loss

    output = denormalize_image(target.detach())
    output = output.clamp(0.0, 1.0)

    return output.detach(), final_content_loss.item(), final_style_loss.item(), final_total_loss.item()
```

**Figure 22c.** *Code implementation of the style_transfer function, part (c)*

```
def show_tensor_image(img_tensor, title=None):
    """
    img_tensor: torch.Tensor in [0,1], shape (1,3,H,W) or (3,H,W) or (H,W,3)
    """
    t = img_tensor.detach().cpu()

    if t.ndim == 4:          # (1,3,H,W)
        t = t[0]
    if t.shape[0] == 3:      # (3,H,W) -> (H,W,3)
        t = t.permute(1, 2, 0)

    t = t.clamp(0, 1).numpy()

    plt.figure(figsize=(6, 6))
    plt.imshow(t)
    plt.axis("off")
    if title:
        plt.title(title)
    plt.show()


def to_display_01(x):
    if x.ndim == 3:
        x = x.unsqueeze(0)
    x01 = denormalize_image(x)
    return x01.clamp(0,1)
```

**Figure 23a.** *Helper functions supporting style transfer - show_tensor_image, to_display_01*

```python
def get_features(image, model, layers):
    features = {}
    x = image
    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[name] = x
    return features


def load_image(img_path, shape=(224, 224)):
    image = Image.open(img_path).convert('RGB')
    # Define transformation to resize, normalize, and convert to tensor
    in_transform = transforms.Compose([
        transforms.Resize(shape),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
    ])
    # Apply transformations, remove alpha channel, and add batch dimension
    image = in_transform(image)[:3, :, :].unsqueeze(0)
    return image.to(device)


def gram_matrix(tensor):
    # tensor: (N, C, H, W)
    N, C, H, W = tensor.size()
    x = tensor.view(N, C, H * W)
    gram = x @ x.transpose(1, 2)    # (N, C, C)
    return gram / (C * H * W)

def denormalize_image(tensor):
    mean = torch.tensor(IMAGENET_MEAN, device=tensor.device).view(1,3,1,1)
    std  = torch.tensor(IMAGENET_STD, device=tensor.device).view(1,3,1,1)
    return tensor * std + mean
```

**Figure 23b.** *Helper functions supporting style transfer - Feature extraction, image loading with ImageNet normalization, Gram matrix computation with $(C \cdot H \cdot W)$ normalization, and denormalization.*

```python
def show_triplet(content, style, output):
    # imgs = [content, style, output]
    imgs = [to_display_01(content), to_display_01(style), output.clamp(0,1)]
    titles = ["Content", "Style", "Output"]

    plt.figure(figsize=(15, 5))
    for i, (img, title) in enumerate(zip(imgs, titles), start=1):
        t = img.detach().cpu()
        if t.ndim == 4:
            t = t[0]
        if t.shape[0] == 3:
            t = t.permute(1, 2, 0)
        t = t.clamp(0, 1).numpy()

        ax = plt.subplot(1, 3, i)
        ax.imshow(t)
        ax.set_title(title)
        ax.axis("off")

    plt.tight_layout()
    plt.show()
```

**Figure 23c.** *Helper functions supporting style transfer - Triplet visualization of content, style, and output.*

## 2.2 Style loss normalization

In Gram-based NST, **style loss normalization is important** because the raw Gram matrix magnitude scales with the number of channels and spatial locations in the feature map. Without normalization, deeper layers (with different $C, H, W$) can dominate the style objective simply due to having larger activation tensors, making the effective balance between layers and between style and content unstable across models and layer choices. In this project, normalization is implemented explicitly in *gram_matrix* by dividing by $(C \cdot H \cdot W)$ (Figure 23b), which keeps Gram values on a more comparable scale across layers and improves optimization stability. In addition, when *style_layer_weights* are provided, the code normalizes them to sum to 1 (Figure 22a), so that the style loss is controlled by relative layer importance rather than the absolute scale of user-provided weights. Together, these two normalizations reduce sensitivity to architecture-specific feature sizes and make the hyperparameters $\alpha$ =*content_weight* and $\beta$ =*style_weight* more interpretable, since changes in $\beta$ correspond more directly to "more stylization pressure" rather than compensating for unnormalized Gram magnitudes.

## 2.3 Style Transfer Process

### 2.3.1 Setup and objective function

The style-transfer tuning stage uses Optuna to search for hyperparameters that reliably make stylized images appear "more Van Gogh" to a fixed, trained classifier. The **judge model** is set to the fine-tuned **VGG-19 classifier from Part 1** (*judge_model* = *vgg_final*), chosen because it achieved the strongest overall minority-class performance (**higher recall and F1**) and therefore

provides a more sensitive signal for "Van Gogh-ness" during tuning. Separately, the **style-transfer backbone** is a pretrained **VGG-19 feature extractor** (*st_model = models.vgg19(pretrained=True)*), used only to compute feature maps and Gram matrices inside *style_transfer*. This separation is intentional: the judge provides the optimization target, while the style-transfer network provides the perceptual representation used to construct content and style losses.

To keep trial comparisons fair and computationally feasible, the objective is evaluated on a **fixed set** of images defined once before the sweep: $N_{\text{CONTENT}} = 10$ content images sampled from **non Van Gogh** paintings in the test dataframe (*test_df[test_df["is_van_gogh"]== False].sample(...)*), and $N_{\text{STYLE}} = 5$ style images chosen as five of Van Gogh's most famous and representative works (The Starry Night, Sunflowers, Irises, Wheatfield with Crows, Almond Tree in Blossom). Sampling **only non Van Gogh** content makes the evaluation meaningful: a high judge probability after stylization indicates that the transfer process successfully injects Van Gogh-like cues into images that were originally negative, rather than simply preserving an already-positive label.

Within each Optuna trial, the code searches three elements: (i) *content_weight* in **[1e-1, 1e3]** (log scale), (ii) *style_weight* in **[1e2, 1e6]** (log scale), and (iii) one positive weight per style layer *w_style_{layer}* in **[1e-3, 1]** (log scale), which is then normalized to sum to 1. The wide log-scale ranges are appropriate for NST because the relative magnitude of content-feature MSE and Gram-based style loss can differ by orders of magnitude, so the useful tradeoff often lies on a multiplicative scale rather than an additive one. Normalizing the style-layer weights ensures the search controls **relative** emphasis across layers (coarse vs fine texture statistics) rather than letting the absolute sum of weights arbitrarily inflate the style loss. For runtime reasons, each candidate is evaluated with a reduced optimization budget (*num_steps=100*), so Optuna can compare many configurations, and the best setting can later be rerun with a larger number of steps for higher-quality outputs. After each NST run, the output is re-normalized for the judge using *preprocess_output_for_classifier*, since the style-transfer function returns an image in $[0,1]$ rather than ImageNet-normalized space.

### 2.3.2 Optimal hyperparameters found

The Optuna sweep identifies **trial 12** as the best configuration for maximizing the judge's "Van Gogh probability" over the fixed evaluation set (10 non Van Gogh content images × 5 Van Gogh style images, with *num_steps=100* per transfer during search). The selected hyperparameters are:

- *content_weight* **($\alpha$)** = 0.104243783015761
- *style_weight* **($\beta$)** = 825429.1199338151
- *style_layer_weights =[0.3653431896, 0.0419071367, 0.2823767234, 0.3103729502],* (*style_layer_weights* are normalized weights for the default VGG style layers ['0','5','10','19'] we defined in the *style_transfer* function)

This outcome is consistent with the practical scale mismatch between the two loss components in NST: Gram-based style losses often require a much larger coefficient than the content feature

MSE to produce visible and classifier-relevant stylization, so a large $\beta$ alongside a moderate $\alpha$ is expected in a setup that is explicitly optimizing for "Van Gogh-ness". The learned layer weighting further suggests that the optimization benefits from emphasizing both very early texture cues (layer '0', the largest weight) and mid-level style statistics (layers '10' and '19'), while assigning relatively little weight to layer '5' in this particular search space and judge objective.

## 2.4 Classifier evaluation of style-transfer outputs

### 2.4.1 Evaluation protocol and visualization format

In Part 2C, the goal is to assess whether neural style transfer can induce a Van Gogh like "signature" on **20 personal content photos**. We apply neural style transfer to each content image using a small set of Van Gogh style paintings (cycled across the 20 contents), and generates **two stylized outputs per content style pair**, one using **AlexNet as the NST feature backbone** and one using **VGG-19 as the NST feature backbone**. Each generated output is then evaluated by **both** trained judges from Part 1 (AlexNet classifier and VGG-19 classifier), producing two probabilities $P(\text{Van Gogh})$ per output. To make the comparison interpretable, we visualize results in a consistent **4 panel layout**: (i) the content photo, (ii) the style painting, (iii) the AlexNet NST output annotated with $P_{\text{AlexNet judge}}$ and $P_{\text{VGG judge}}$, and (iv) the VGG19 NST output annotated with the same two probabilities. Figures 24–26 provide representative examples of this evaluation, including cases of agreement and disagreement between the two judges and between the two NST backbones.

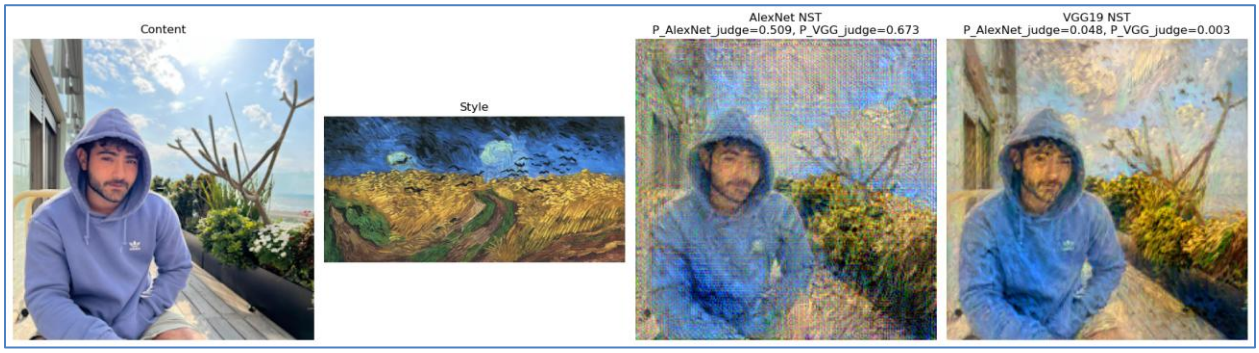


**Figure 24.** *Agreement example- AlexNet NST output is classified as 'Van Gogh' by both judges, and VGG NST output is classified as 'non Van Gogh' by both judges.*

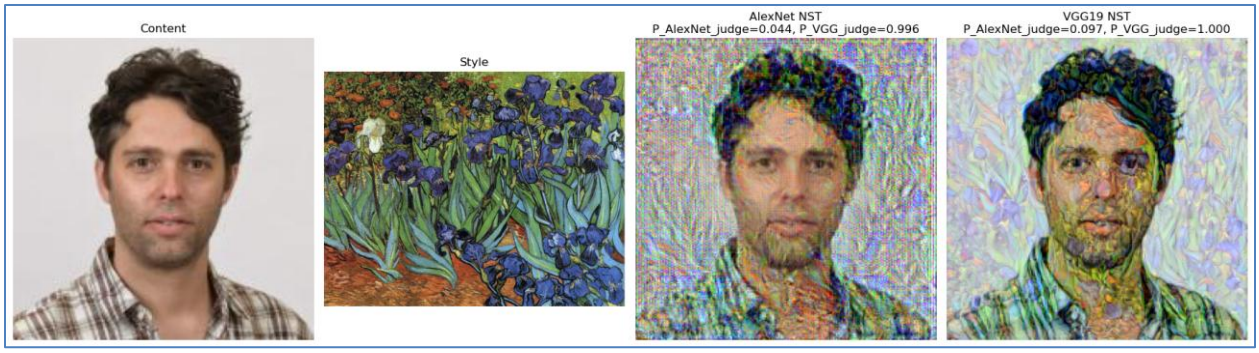


**Figure 25.** *Disagreement example- For both AlexNet NST output and VGG NST output, the AlexNet judge classifies as 'non Van Gogh', and the VGG judge as 'Van Gogh'.*


**Figure 26.** *Combined example- AlexNet NST output classified as 'non Van Gogh' by AlexNet judge, and as 'Van Gogh' by VGG judge, whereas VGG NST output is classified as 'non Van Gogh' by both judges.*

### 2.4.2 Analysis of outputs differences and judge responses

Figures 24–26 reveal several consistent patterns when comparing the two NST backbones (AlexNet vs. VGG-19) and the two trained judges (AlexNet classifier and VGG-19 classifier). Overall, the examples show that (i) the **NST backbone** mainly controls how strongly and where the Van Gogh texture is injected, while (ii) the **judge architecture** controls which visual cues are treated as "Van Gogh evidence", sometimes producing judge disagreements.

**(I) Hyperparameter selection and fairness of comparison:**

The NST hyperparameters reported in this section (content/style weights and layer-weight configuration) were selected via Optuna using the VGG-19 NST backbone, and then applied unchanged when generating AlexNet-NST outputs. Because the style and content losses are computed in the backbone's feature space, the optimal tradeoff is backbone-dependent. Therefore, this protocol may favor VGG19-NST and place AlexNet-NST at a disadvantage (e.g., more noise or less coherent brushstroke structure). As a result, part of the observed differences between AlexNet-NST and VGG19-NST in Figures 24–26 may reflect this tuning asymmetry rather than an inherent limitation of the AlexNet backbone. We treat this as a limitation of the comparison, and a fully controlled study would tune NST hyperparameters separately for each backbone or perform a shared tuning procedure that optimizes performance for both.

**(II) Backbone-dependent "strength vs. structure" tradeoff**

AlexNet-NST tends to be noisier, while VGG19-NST tends to be more coherent.
Across the examples, AlexNet-based NST frequently produces a more aggressive, grainy texture

overlay that can partially wash out semantic structure (faces, limbs, object boundaries), whereas VGG19-based NST usually preserves the content structure more cleanly while still introducing smoother brush-like patterns. This can be seen clearly in the stylized portraits in Figures 24–25, where the VGG19-NST output maintains more consistent edges and facial geometry, while the AlexNet-NST output often looks more "textured" but also more distorted. This difference is consistent with the expectation that a deeper backbone (VGG-19) provides a richer multi-scale representation that can guide stylization without collapsing content structure as quickly.

**(III) texture-biased vs. structure-biased evidence**

Judge disagreement indicates different "Van Gogh cues" learned by the classifiers**.**

A key observation is that the two judges do not always agree on whether a stylized image looks Van Gogh-like, even when they score the same generated output. Figure 26 is the clearest demonstration: for the AlexNet-NST output, the **VGG-judge assigns a very high probability** $P_{\text{VGG judge}} = 0.974$, while the **AlexNet-judge remains low** $P_{\text{AlexNet judge}} = 0.091$. This suggests that the VGG-judge is more sensitive to the **global brushstroke statistics and texture correlations** that resemble Van Gogh's style (which the NST procedure explicitly optimizes via Gram matrices), while the AlexNet-judge may require additional cues (for example, more coherent Van Gogh-like edges, color harmonies, or less noisy artifacts) before it raises confidence. In other words, the same stylization can be interpreted as "strong Van Gogh signal" by one judge and "insufficient or noisy style" by the other.

**(IV) Content complexity can increase judge disagreement**

Figure 26 also highlights that highly complex scenes (crowds, strong lighting contrast, many small objects and edges) can create stylized outputs where the style signal becomes fragmented across the image. In such cases, Gram-matrix-driven texture injection may produce many local brush-like patterns, but the overall composition remains hard to interpret. This can explain why one judge (VGG) responds strongly to the presence of distributed texture statistics, while the other judge (AlexNet) does not, potentially because the stylization introduces artifacts that interfere with the cues it relies on. Practically, this means that "success" in NST is not only about producing a visually pleasing output, but also depends on whether the stylized result aligns with the specific feature sensitivities of the trained Van Gogh detector.

**(V) Summary**

Figures 24–26 support the conclusion that VGG19-based NST more consistently preserves content structure while transferring style, whereas AlexNet-based NST often produces stronger but noisier stylization. At the same time, the learned judges are not interchangeable: the VGG-judge tends to reward texture statistics consistent with the style loss, while the AlexNet-judge can be more conservative, especially when stylization artifacts or complex content make the "Van Gogh signature" less coherent.

### 2.5 Layer selection and justification

Our NST implementation extracts intermediate feature maps from the **convolutional feature extractor** (*model = model.features*) and defines:

- **Content representation** as the activation tensor at a selected **deep** layer.
- **Style representation** as **Gram matrices** of activation tensors at several **earlier-to-mid** layers (capturing texture statistics at multiple spatial scales).

This choice is implemented directly in *style_transfer* via the default layer lists that are used when *content_layers* and *style_layers* are not provided.

#### 2.5.1 AlexNet selected layers

In our AlexNet-based NST, we use:

- **Style layers:** ['0', '3', '6', '8']
  These correspond to AlexNet **Conv1, Conv2, Conv3, Conv4** in *alexnet.features*.
- **Content layer:** ['10']
  This corresponds to **Conv5**.

AlexNet is relatively shallow, so Conv5 is the most suitable "high-level" feature map available for preserving the overall spatial arrangement of the content (rough pose and global structure), while Conv1-Conv4 progressively capture richer textures and patterns that can be matched to the Van Gogh style via Gram statistics.

#### 2.5.2 VGG-19 selected layers

In our VGG-based NST, we use:

- **Style layers:** ['0', '5', '10', '19']
  These correspond approximately to the first conv layer in each early block: **Conv1_1, Conv2_1, Conv3_1, Conv4_1** in *vgg19.features*.
- **Content layer:** ['21']
  This corresponds to **Conv4_2**, a common and effective content layer in VGG-based NST.

VGG-19 has a deeper hierarchy and more stable feature statistics for texture synthesis. Using Conv4_2 as the content layer typically preserves scene layout and object boundaries well, while using multiple style layers from shallow to mid-depth captures style at multiple granularities (colors and small brush strokes in early layers, thicker textures and repeated motifs in deeper layers).

#### 2.5.3 Why shallow layers for style and deep layers for content

The layer choice follows the standard separation between "what" and "how it looks":

- **Shallow layers (early convs)** have small receptive fields and respond strongly to **edges, local color contrasts, and fine textures**. Matching Gram matrices at these layers encourages the target image to reproduce Van Gogh-like **brushstroke micro-texture** and palette statistics.
- **Deeper layers** have larger receptive fields and encode more **global structure** (spatial arrangement and higher-level patterns). Enforcing content similarity at a deeper layer constrains the target to maintain the **composition** of the content photo while still allowing surface appearance to change.

Using multiple style layers is important because "style" is not a single-scale phenomenon. In our code, the total style loss sums per-layer Gram-MSE terms weighted by *style_layer_weights* (normalized to sum to 1), so we can balance how much each scale contributes.

### 2.5.4 Implications for the observed results

This layer design also explains typical behavior we analyzed in Figures 24–26:

- AlexNet (shallower, fewer feature channels and blocks) often produces **coarser or noisier textures** when pushed strongly toward style, because it has less representational depth to separate structure from texture cleanly.
- VGG-19 typically yields **more coherent brushstroke patterns** and smoother stylization because its deeper hierarchy supports richer multi-scale texture constraints without collapsing the content structure as quickly.

## 2.6 Overall Reflection

The Part 2 findings align closely with the Part 1 error patterns: both classifiers, and especially VGG-19, appear highly responsive to texture and brushstroke statistics rather than purely semantic content. In Part 1, many false positives were non Van Gogh paintings that nonetheless exhibit strong, stroke-like textures and saturated contrasts, while some false negatives were Van Gogh works with weaker or atypical high-frequency cues. In Part 2, neural style transfer explicitly injects exactly these texture statistics via Gram-matrix matching, so it is not surprising that stylized outputs can shift the classifier's decision even when the underlying scene remains unchanged. This mechanism is visible in Figures 24–26, where stronger or more coherent texture transfer tends to increase the VGG-judge's "Van Gogh" probability, while the AlexNet-judge is often more conservative when the stylization is noisy or disrupts structure. In other words, Part 2 provides a controlled intervention that supports the Part 1 interpretation: "Van Gogh-ness" for these models is largely encoded as transferable texture and color statistics, and the observed AlexNet vs VGG differences reflect different sensitivities to these cues.

A second implication of this alignment is that classifier based "Van Gogh probability" should be interpreted as a **proxy for learned stylistic cues**, not as a complete measure of NST quality. In Part 1, the existence of false positives indicates that non Van Gogh works containing dense, high frequency texture and saturated contrasts can trigger the learned decision rule. Part 2 effectively injects similar statistics by construction through Gram based style matching. Figures 24–26 reinforce this point: an output may receive a high score from one judge and a low score from the other, meaning that different architectures treat different texture patterns as evidence of Van Gogh. Moreover, because the NST hyperparameters were tuned using the VGG based NST setup, the qualitative head to head between AlexNet NST and VGG19 NST is not perfectly symmetric and may partially reflect tuning in VGG's feature space rather than an intrinsic advantage of VGG19 NST. Overall, Part 2 strengthens the Part 1 conclusion that both models rely heavily on transferable texture and color statistics, but it also motivates evaluating NST outputs with both qualitative inspection and agreement across judges, rather than relying on a single classifier score as the sole criterion of success.

## Part 3: GPU vs. CPU Benchmark

As shown in Figure 27, PyTorch reports *cuda available: True* and identifies the GPU as **NVIDIA GeForce RTX 4090**. The same cell logs the Python version and machine identifier, and shows 0.0 MB allocated/reserved at the beginning of execution, meaning the process starts with an empty CUDA memory state before the training loop begins.
To compare hardware fairly, we measured the wall clock training time required to complete one epoch using the same model, dataset, batch size, and training loop, once with *device="cuda"* **(GPU)** and once with *device="cpu"* **(CPU)**.
The results are summarized in Figure 29. The **GPU epoch time** is **204.739** seconds, while the **CPU epoch time** is **215.475** seconds, a difference of **10.736** seconds (about a **5%** speedup) in favor of the GPU. While the GPU is faster, the improvement is smaller than what we expected.
A likely reason for the small gap is that one epoch includes more than just "GPU math". Parts like loading images from disk and applying preprocessing/augmentations are still done on the CPU, so they can limit the overall speedup. Also, if the batch size is relatively small, the GPU may not be fully utilized all the time, which reduces the advantage over CPU. Crucially, the modest speedup is not because the GPU was idle. The *nvidia-smi* screenshot in Figure 28 shows the training process actively using the GPU, with GPU utilization around **81%** and roughly **6.47 GB** of **GPU memory** in use (out of **24 GB**), and the running Python process listed under GPU processes. This confirms that the model executed on the GPU as intended.
Overall, the GPU advantage is expected to become more pronounced in heavier workloads (larger batches, more compute intensive backbones, reduced preprocessing overhead, or longer iterative optimization such as style transfer), whereas in an end to end single epoch timing, pipeline costs can dominate and shrink the observed speedup.

```
print("cuda available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU Device:", torch.cuda.get_device_name(0))
print("Python version:", platform.python_version())
print("Machine name:", platform.node()) # This is unique to the machine
print("Working directory:", os.getcwd())
if torch.cuda.is_available():
    print("Initial GPU memory usage:")
    print(torch.cuda.memory_allocated() / 1024**2, "MB allocated")
    print(torch.cuda.memory_reserved() / 1024**2, "MB reserved")
```

✓ 0.0s

```
cuda available: True
GPU Device: NVIDIA GeForce RTX 4090
Python version: 3.12.10
Machine name: wol08-424-pc
Working directory: w:\IDL_Final_Project\IDL_Final_Project
Initial GPU memory usage:
0.0 MB allocated
0.0 MB reserved
```

**Figure 27.** *Environment and device verification from the notebook output.*

```
(updated_python_venv) PS W:\IDL_Final_Project\IDL_Final_Project> nvidia-smi
Sun Jan 11 16:08:10 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.97                 Driver Version: 580.97         CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                  Driver-Model | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 4090      WDDM  |   00000000:01:00.0 Off |                  Off |
|  0%   44C    P3             53W /  450W |    6470MiB /  24564MiB |     81%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            9108      C   ...am Files\Python312\python.exe      N/A      |
+-----------------------------------------------------------------------------------------+
(updated_python_venv) PS W:\IDL_Final_Project\IDL_Final_Project>
```

**Figure 28.** *nvidia-smi screenshot during training, showing GPU model, memory usage, and utilization*

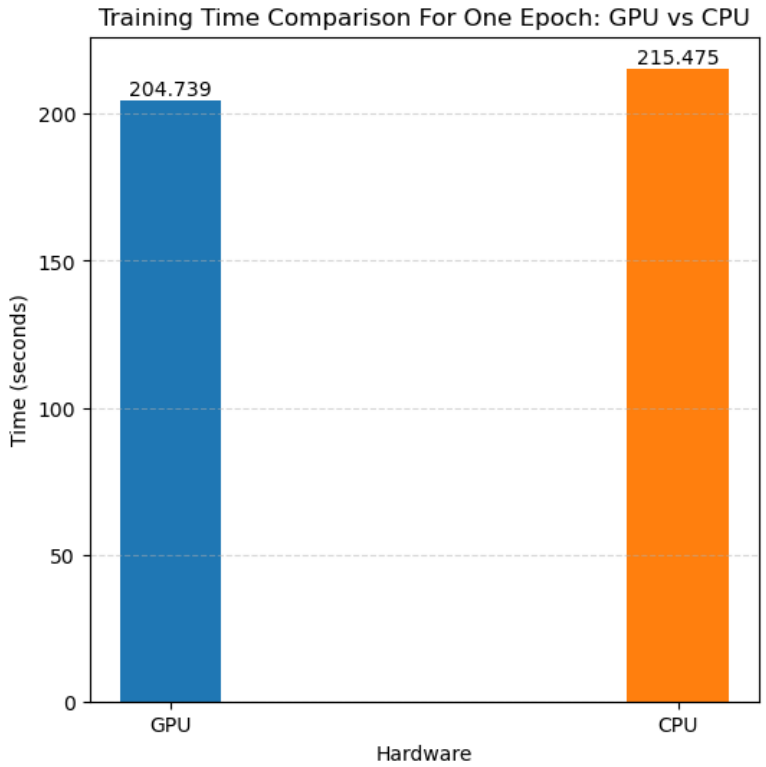


**Figure 29.** *Training time comparison for one epoch (GPU vs. CPU)*

## Part 4: Model interpretability with Grad-CAM (Bonus)

### 4.1 Bonus Part overview

Grad-CAM is a post hoc interpretability method that highlights which image regions most influenced the classifier's decision by backpropagating gradients from the target class to a chosen convolutional layer and projecting them to the input space. In this bonus, we use Grad-CAM to move beyond "what label was predicted" and analyze "why", by visualizing where each judge attends when it predicts **Van Gogh**. This lets us compare what the AlexNet-based judge and the VGG-based judge treat as a "Van Gogh signature", and to diagnose systematic failure modes using False Positives and confirmed cues using True Positives. The results are summarized in Figures **30–33**

### 4.2 What the model uses when it is correct (True Positives)

Figures **30** (AlexNet TP) and **31** (VGG TP) show cases where the classifier correctly predicts Van Gogh with high confidence. Across these examples, the strongest activations tend to fall on **textural** regions rather than semantic objects: dense brush-stroke patterns, high-frequency edges, and areas with strong local contrast. This suggests that "Van Gogh-ness" is learned primarily as a **texture and stroke statistic**, not as a particular object category.

Comparing the two judges, the attention is broadly consistent but not identical. AlexNet's heatmaps are typically **coarser and more blob-like**, often covering larger areas that contain repeated texture. VGG's behavior is typically **more spatially selective**, concentrating on thinner bands or patches where the stroke pattern is strongest (for example, along prominent boundaries and highly textured regions). In both cases, the attention rarely looks like it is "looking for faces" as a special cue. When faces appear in the image, the highlighted regions usually track **hair-like texture, clothing folds, or background texture** more than facial landmarks, which supports the interpretation that the learned signal is dominated by **brushwork-like texture** rather than "portrait content".

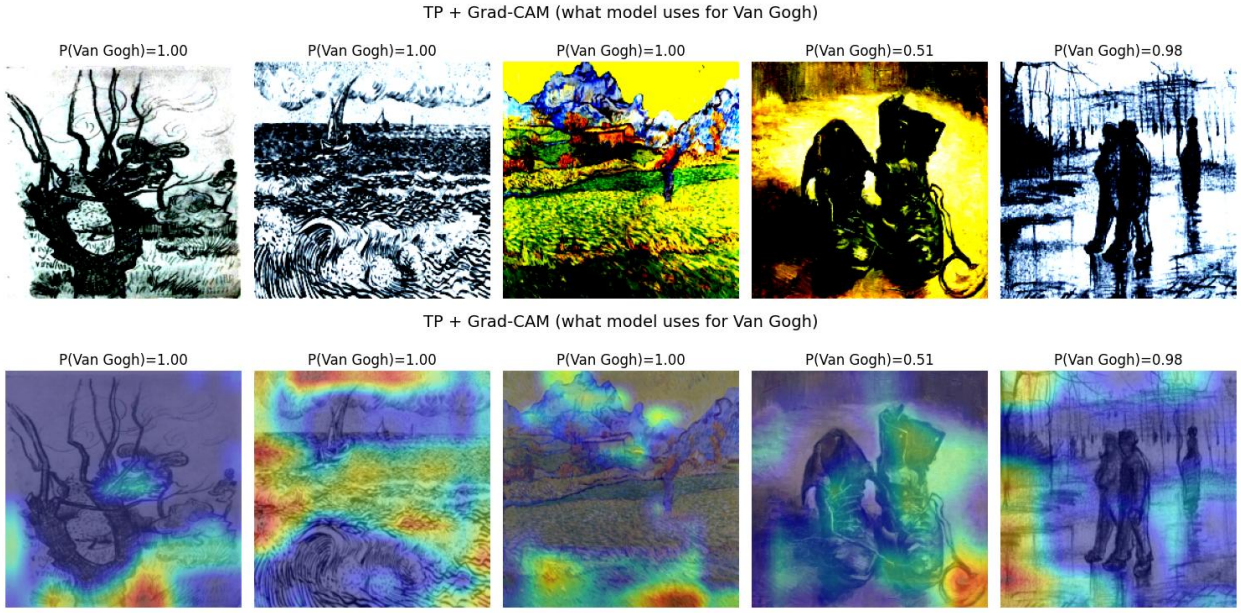


**Figure 30.** *AlexNet TP pictures and heatmap*

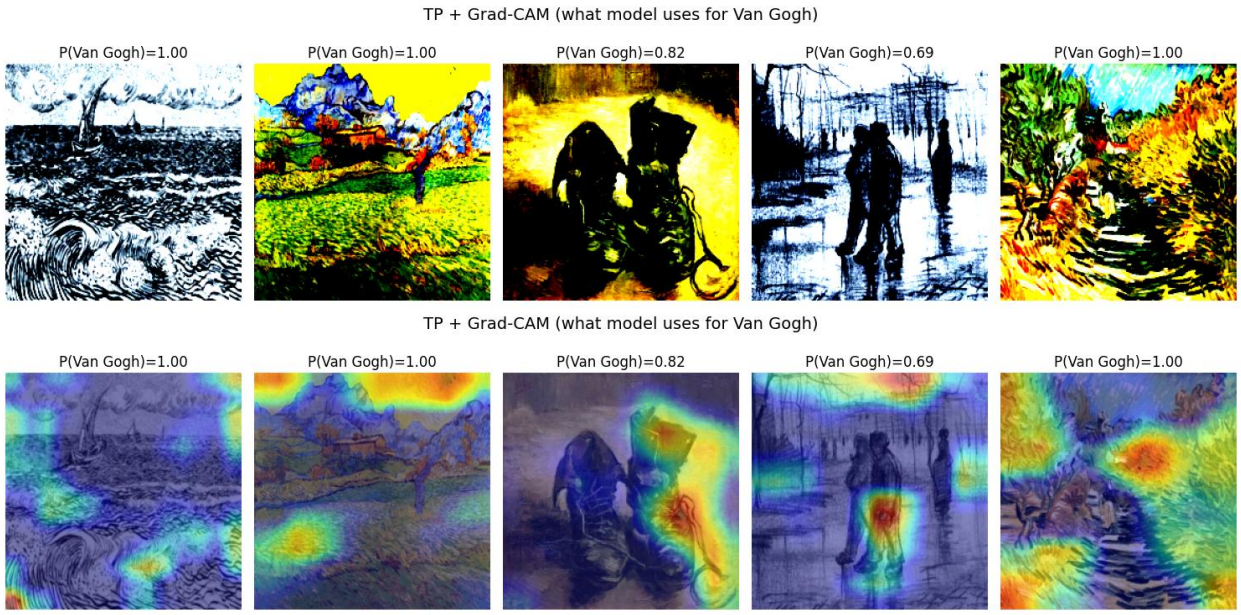


**Figure 31.** *VGG TP heatmap*

## 4.3 What fooled the model (False Positives)

Figures **32** (AlexNet FP) and **33** (VGG FP) show non–Van Gogh images that were nevertheless classified as Van Gogh with high confidence. These cases are especially informative because they reveal what patterns can **mimic** the Van Gogh signature for the model. A recurring pattern is that the heatmaps focus on regions that contain **Van Gogh-like visual statistics** even when the global content is unrelated: swirly or turbulent sky-like textures, wave foam or repetitive short strokes in landscapes, dense foliage textures, and strongly textured "hair-like" regions.

A representative error pattern is that the model can treat a **swirly cloud-like texture** (or any similar repeated curved stroke pattern) in another artist's landscape as evidence for Van Gogh. Another frequent trigger is **high-contrast, repetitive linework** and "sketch-like" structure, where the model locks onto concentrated edge density and interprets it as the kind of structured mark-making it has seen in Van Gogh examples. Between architectures, AlexNet tends to over-rely on **large, high-contrast textured patches** (more global and less precise), while VGG often localizes to **more specific texture motifs**, which can reduce some spurious triggers but can also make it very sensitive to small regions that incidentally match the learned stroke statistics.

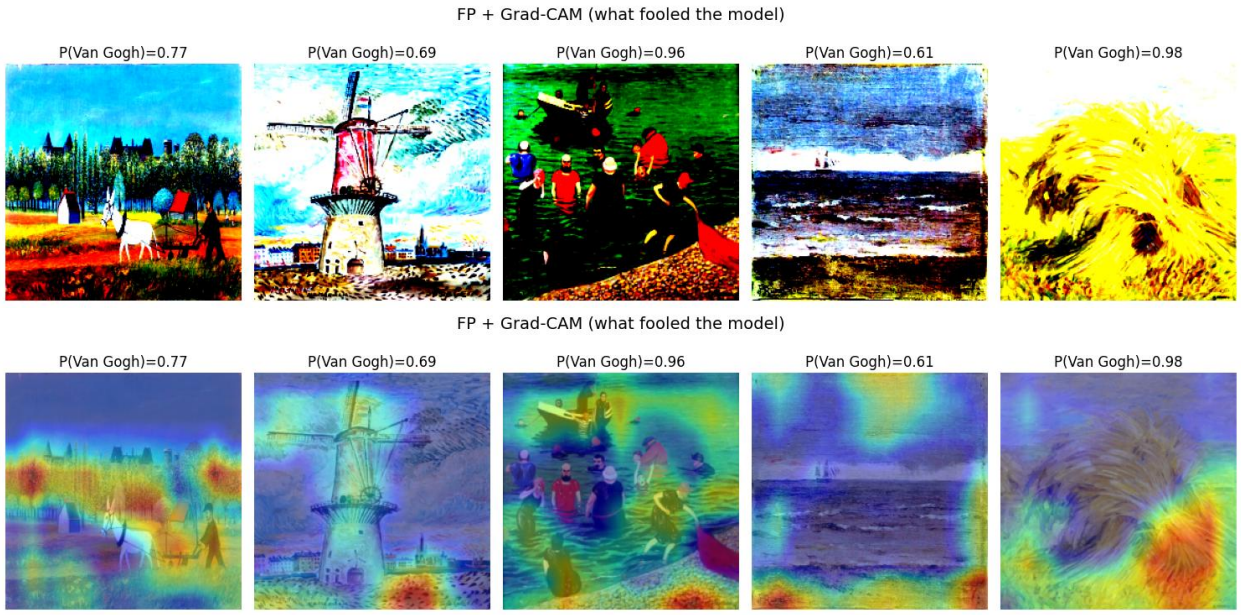


**Figure 32.** *AlexNet FP heatmap*

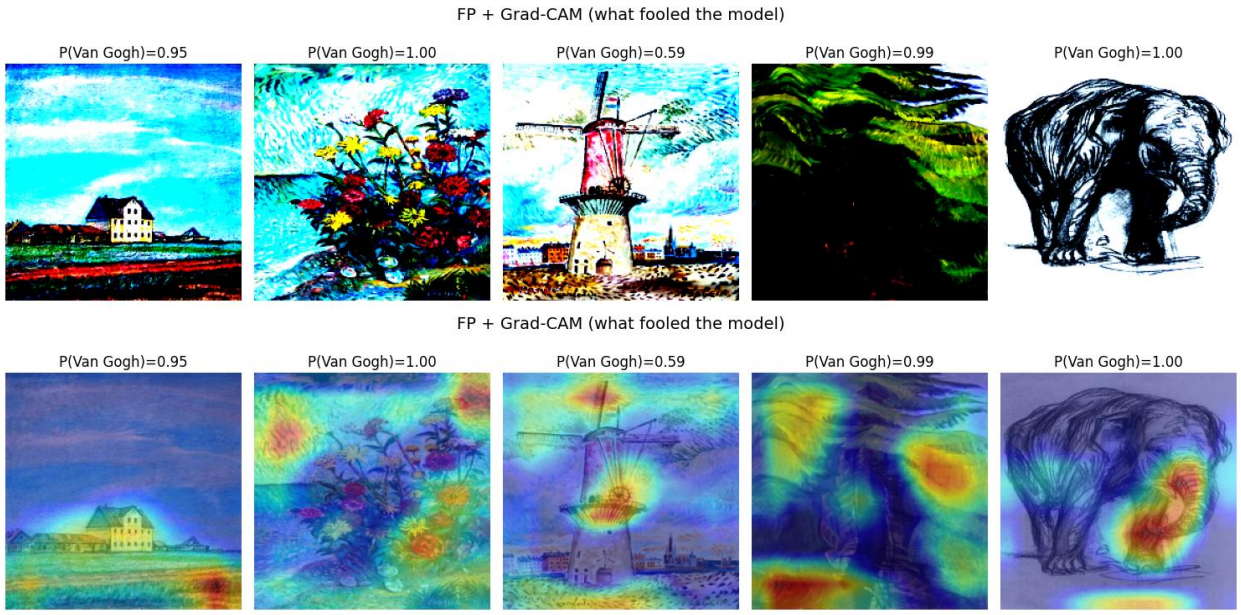


**Figure 33.** *VGG FP heatmap*

## 4.4 Conclusions: what "visual signature" did the classifier learn?

Taken together, the TP and FP Grad-CAM maps suggest that the classifiers learned a "Van Gogh signature" that is dominated by **local texture cues** rather than object-level semantics. The most consistent cues are:

(i) **directional, short brush strokes** that create a high-frequency texture field.
(ii) **curved or swirling stroke arrangements** that produce a distinctive flow pattern.
(iii) **strong local contrast and edge density** that resembles energetic mark-making.

(iv) recurring **color-texture combinations** that appear often in the training distribution (for example, highly saturated regions or characteristic contrasts).

This explains why some non–Van Gogh landscapes, drawings, or highly textured scenes can fool the model: if a small region matches these texture statistics, the model can predict Van Gogh even when the overall scene is unrelated.

### 4.5 Using Grad-CAM to improve Part 2 style transfer

In Part 2, the style loss is applied uniformly over the full image, which can over-stylize important content (especially faces) and reduce identity or structure preservation. Grad-CAM offers a practical way to make style transfer **spatially aware** by using the heatmap as a weight mask. We can compute a Grad-CAM heatmap $H(x)$ on the **content image** and normalize it to $[0,1]$. Then define a mask that protects important regions, for example:

$$M_{\text{protect}} = H \ , \ M_{\text{style}} = 1 - H$$

and modify the losses so that style is emphasized where $M_{\text{style}}$ is large and de-emphasized where $M_{\text{protect}}$ is large:

- **Masked style loss (protect salient regions):** compute the style loss using only the regions selected by $M_{\text{style}}$(background), for example by applying the mask to feature maps before computing Gram matrices (or by computing Gram matrices on masked spatial features).
- **Masked content loss (preserve salient regions):** increase content preservation where $M_{\text{protect}}$ is high, for example by weighting the per-pixel (or per-spatial-location) content loss so that the optimization prioritizes structure in salient areas.

The benefit is that style transfer becomes **targeted** instead of uniform. If the heatmap aligns with regions we want to keep realistic (often faces, hands, or other identity-defining areas), those regions receive stronger content protection and weaker style pressure, while the background can carry more stylization. Conversely, if the goal is to maximize Van Gogh "signature", we can invert the logic and **amplify style** precisely in the regions Grad-CAM identifies as most diagnostic for Van Gogh, potentially increasing the judge's Van Gogh probability without destroying the entire content image.